

PROGRAM :

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~
cd OSSP
mkdir -p Program21
cd Program21
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program21$ pwd
/home/vedsk0831/OSSP/Program21
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program21$ nano program21.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program21$ gcc -Wall -Wextra program21.c -o program21
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program21$ ls -l
total 24
-rwxr-xr-x 1 vedsk0831 vedsk0831 16888 Oct  6 13:52 program21
-rw-r--r-- 1 vedsk0831 vedsk0831  3891 Oct  6 13:51 program21.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program21$ ./program21
=====
  OSSP PROGRAM 21
  Integration and Error Handling
=====
Type Linux commands.
Type 'help' for available commands.
Type 'exit' to quit.

shell> echo hello
hello
shell> ls -l
total 24
-rwxr-xr-x 1 vedsk0831 vedsk0831 16888 Oct  6 13:52 program21
-rw-r--r-- 1 vedsk0831 vedsk0831  3891 Oct  6 13:51 program21.c
shell> abcd123
ERROR: Cannot execute 'abcd123': No such file or directory
Command exited with status 127
shell>
ERROR: Empty command.
shell> help
```

```

shell> echo hello
hello
shell> ls -l
total 24
-rwxr-xr-x 1 vedsk0831 vedsk0831 16888 Oct  6 13:52 program21
-rw-r--r-- 1 vedsk0831 vedsk0831 3891 Oct  6 13:51 program21.c
shell> abcd123
ERROR: Cannot execute 'abcd123': No such file or directory
Command exited with status 127
shell>
ERROR: Empty command.
shell> help

Available commands:
  help - display help
  exit - exit shell
  Any Linux command

shell> help

Available commands:
  help - display help
  exit - exit shell
  Any Linux command

shell> abcd123
ERROR: Cannot execute 'abcd123': No such file or directory
Command exited with status 127
shell> echo "shell recovered"
"shell recovered"
shell> |

```

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <errno.h>
```

```
#define MAX_INPUT 256
```

```
#define MAX_ARGS 64
```

```
/* Display error message */
```

```
void print_error(const char *message)
```

```
{
```

```

    fprintf(stderr, "ERROR: %s\n", message);
}

/* Parse command */
int parse_command(char *input, char *args[])
{
    int argc = 0;

    char *token = strtok(input, " ");

    while (token != NULL && argc < MAX_ARGS - 1) {

        args[argc++] = token;

        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    return argc;
}

/* Execute command */
int execute_command(char *args[])
{
    pid_t pid = fork();

    if (pid < 0) {

```

```
    perror("fork");  
    return -1;  
}
```

```
/* Child */
```

```
if (pid == 0) {
```

```
    execvp(args[0], args);
```

```
/*
```

```
 * execvp returns only when an error occurs.
```

```
*/
```

```
fprintf(stderr,
```

```
    "ERROR: Cannot execute '%s': %s\n",
```

```
    args[0],
```

```
    strerror(errno));
```

```
    exit(127);
```

```
}
```

```
/* Parent */
```

```
else {
```

```
    int status;
```

```
    if (waitpid(pid, &status, 0) < 0) {
```

```
    perror("waitpid");  
    return -1;  
}  
  
if (WIFEXITED(status)) {  
  
    int exit_status = WEXITSTATUS(status);  
  
    if (exit_status != 0) {  
  
        printf("Command exited with status %d\n",  
            exit_status);  
    }  
  
    return exit_status;  
}  
  
if (WIFSIGNALED(status)) {  
  
    printf("Command terminated by signal %d\n",  
        WTERMSIG(status));  
  
    return -1;  
}  
}  
  
return 0;  
}
```

```

int main()
{
    char input[MAX_INPUT];

    printf("=====\n");
    printf(" OSSP PROGRAM 21\n");
    printf(" Integration and Error Handling\n");
    printf("=====\n");

    printf("Type Linux commands.\n");
    printf("Type 'help' for available commands.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {

        printf("shell> ");
        fflush(stdout);

        /*
         * Read user input
         */
        if (fgets(input,
            sizeof(input),
            stdin) == NULL) {

            printf("\nInput error or EOF.\n");
            break;

```

```
}
```

```
/*
```

```
 * Remove newline
```

```
*/
```

```
input[strcspn(input, "\n")] = '\0';
```

```
/*
```

```
 * Empty command
```

```
*/
```

```
if (strlen(input) == 0) {
```

```
    printf("ERROR: Empty command.\n");
```

```
    continue;
```

```
}
```

```
/*
```

```
 * Exit
```

```
*/
```

```
if (strcmp(input, "exit") == 0) {
```

```
    printf("Exiting safely...\n");
```

```
    break;
```

```
}
```

```
/*
```

```
 * Help
```

```
*/
```

```

if (strcmp(input, "help") == 0) {

    printf("\nAvailable commands:\n");
    printf("  help - display help\n");
    printf("  exit - exit shell\n");
    printf("  Any Linux command\n\n");

    continue;
}

/*
 * Copy input because strtok()
 * modifies the string.
 */
char command_copy[MAX_INPUT];

strcpy(command_copy, input);

/*
 * Argument array
 */
char *args[MAX_ARGS];

/*
 * Parse command
 */
int argc = parse_command(command_copy, args);

```

```

/*
 * Validate parser output
 */
if (argc == 0) {

    print_error("Invalid command syntax.");
    continue;
}

/*
 * Execute command
 */
int result = execute_command(args);

/*
 * Runtime error handling
 */
if (result == -1) {

    print_error(
        "Command execution failed."
    );
}
}

printf("Program 21 terminated.\n");

return 0;

```

}